

AI ASSISTANT CODING

ASSIGNMENT-4

Name: P. Niharika

Batch:24

Ht. No:2303A51694

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

Steps:

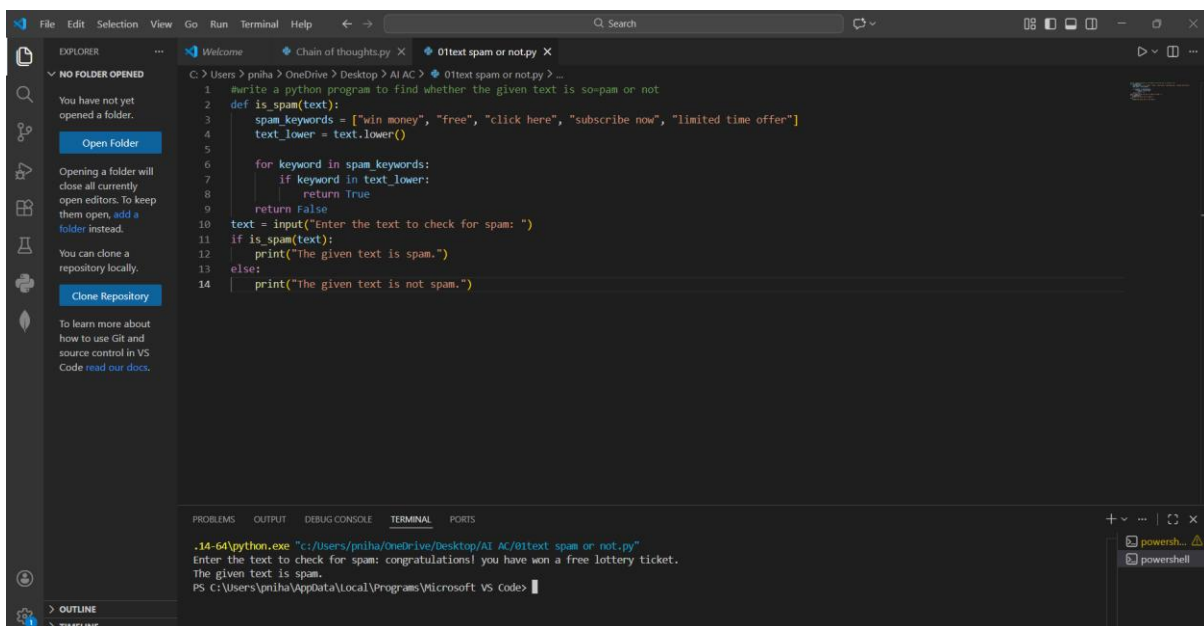
1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."

Expected Output:

Spam



The screenshot shows a Visual Studio Code editor window. The Explorer pane on the left shows 'NO FOLDER OPENED'. The main editor area has two tabs: 'Chain of thoughts.py' and '01text spam or not.py'. The '01text spam or not.py' tab is active, displaying the following Python code:

```
1 #write a python program to find whether the given text is so-pam or not
2 def is_spam(text):
3     spam_keywords = ["win money", "free", "click here", "subscribe now", "limited time offer"]
4     text_lower = text.lower()
5
6     for keyword in spam_keywords:
7         if keyword in text_lower:
8             return True
9     return False
10
11 text = input("Enter the text to check for spam: ")
12 if is_spam(text):
13     print("The given text is spam.")
14 else:
15     print("The given text is not spam.")
```

The bottom panel shows the TERMINAL output:

```
.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/AI AC/01text spam or not.py"
Enter the text to check for spam: congratulations! you have won a free lottery ticket.
The given text is spam.
PS C:\Users\pniha\AppData\Local\Programs\Microsoft VS Code>
```

Q2. One-Shot Prompting (Emotion detection)

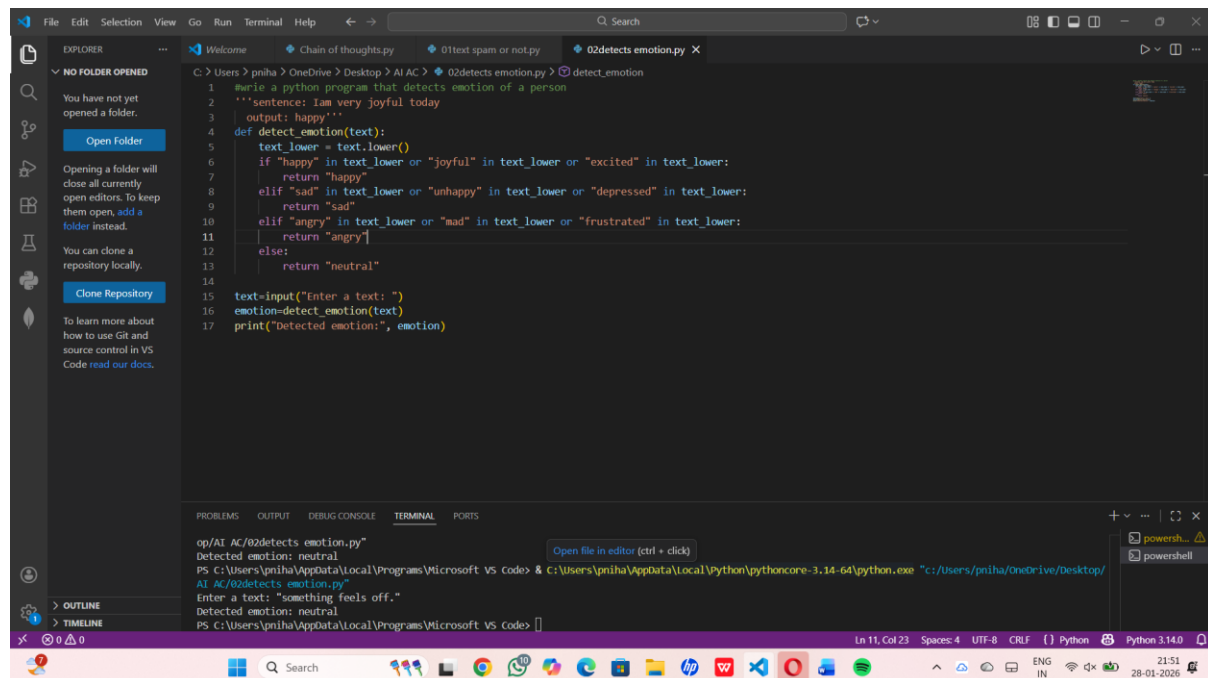
Task:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion



The screenshot shows a Visual Studio Code editor with a Python file named '02detects emotion.py'. The code defines a function 'detect_emotion(text)' that uses a series of if-elif-else statements to determine the emotion of a given text. The emotions are 'happy', 'sad', 'angry', 'excited', 'nervous', and 'neutral'. The program prompts the user to enter a text and prints the detected emotion. The terminal output shows the program being run, the user entering 'something feels off.', and the program outputting 'Detected emotion: neutral'.

```
1 #write a python program that detects emotion of a person
2 '''sentence: Iam very joyful today
3 output: happy'''
4 def detect_emotion(text):
5     text_lower = text.lower()
6     if "happy" in text_lower or "joyful" in text_lower or "excited" in text_lower:
7         return "happy"
8     elif "sad" in text_lower or "unhappy" in text_lower or "depressed" in text_lower:
9         return "sad"
10    elif "angry" in text_lower or "mad" in text_lower or "frustrated" in text_lower:
11        return "angry"
12    else:
13        return "neutral"
14
15    text=input("Enter a text: ")
16    emotion=detect_emotion(text)
17    print("Detected emotion:", emotion)
```

Terminal Output:

```
op\AI AC\02detects emotion.py
Detected emotion: neutral
PS C:\Users\pniha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/AI AC/02detects emotion.py"
Enter a text: "something feels off."
Detected emotion: neutral
PS C:\Users\pniha\AppData\Local\Programs\Microsoft VS Code>
```

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

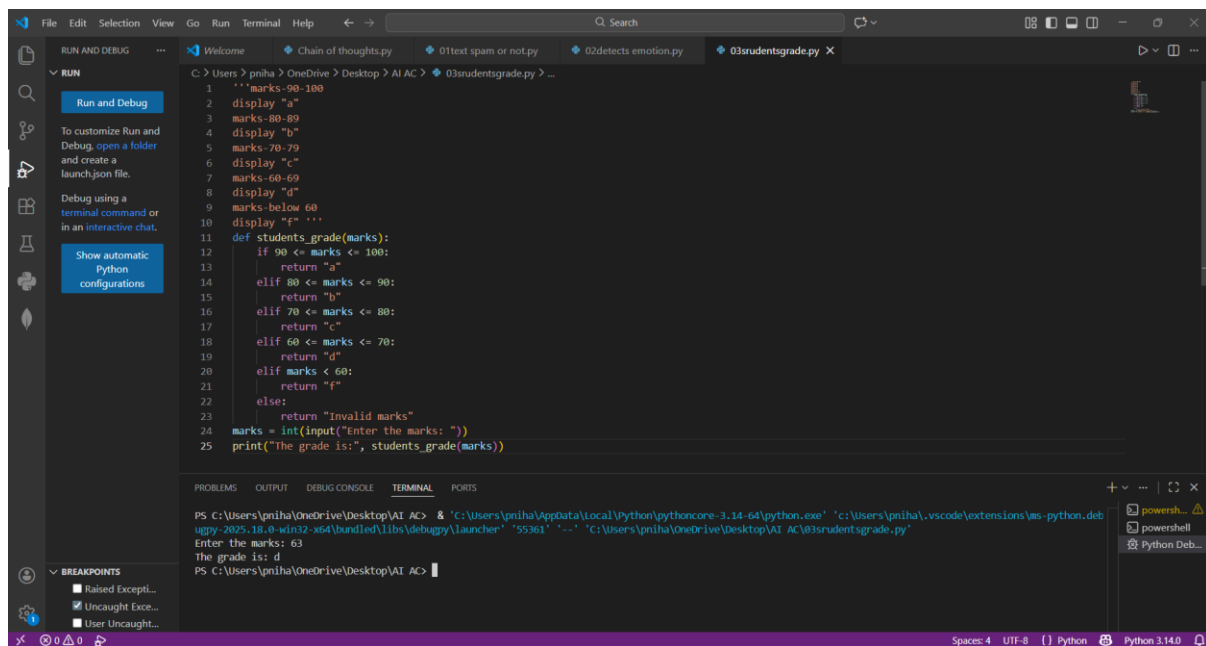
Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F



Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

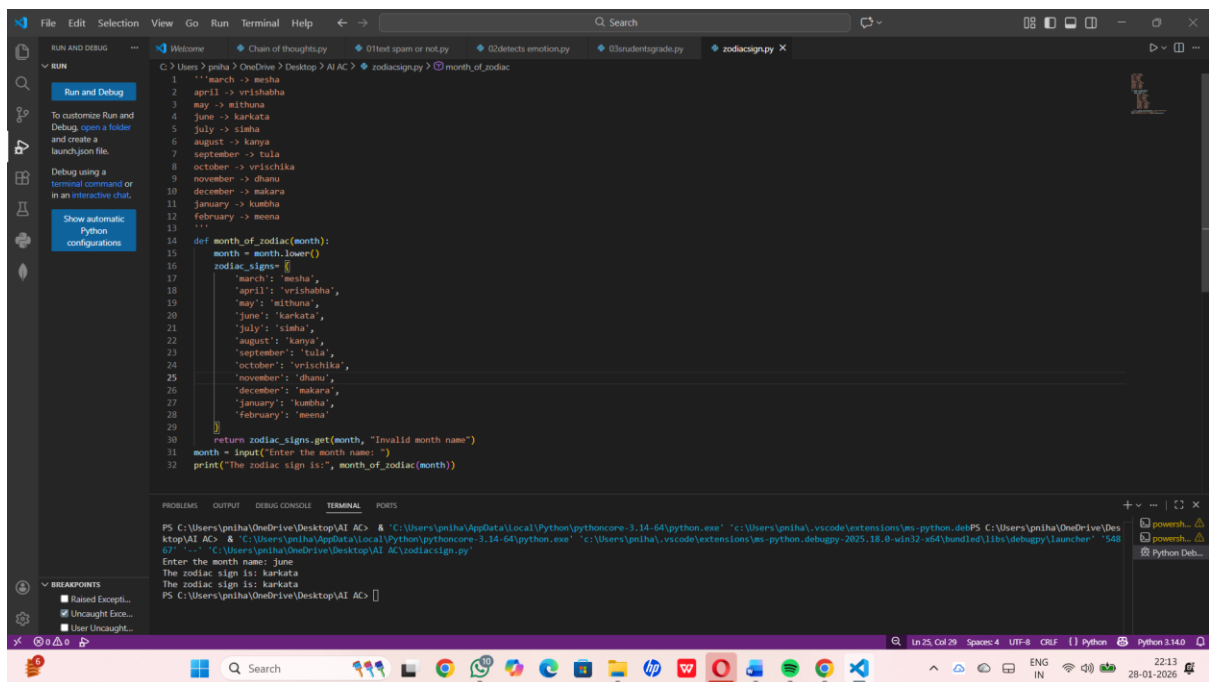
October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena

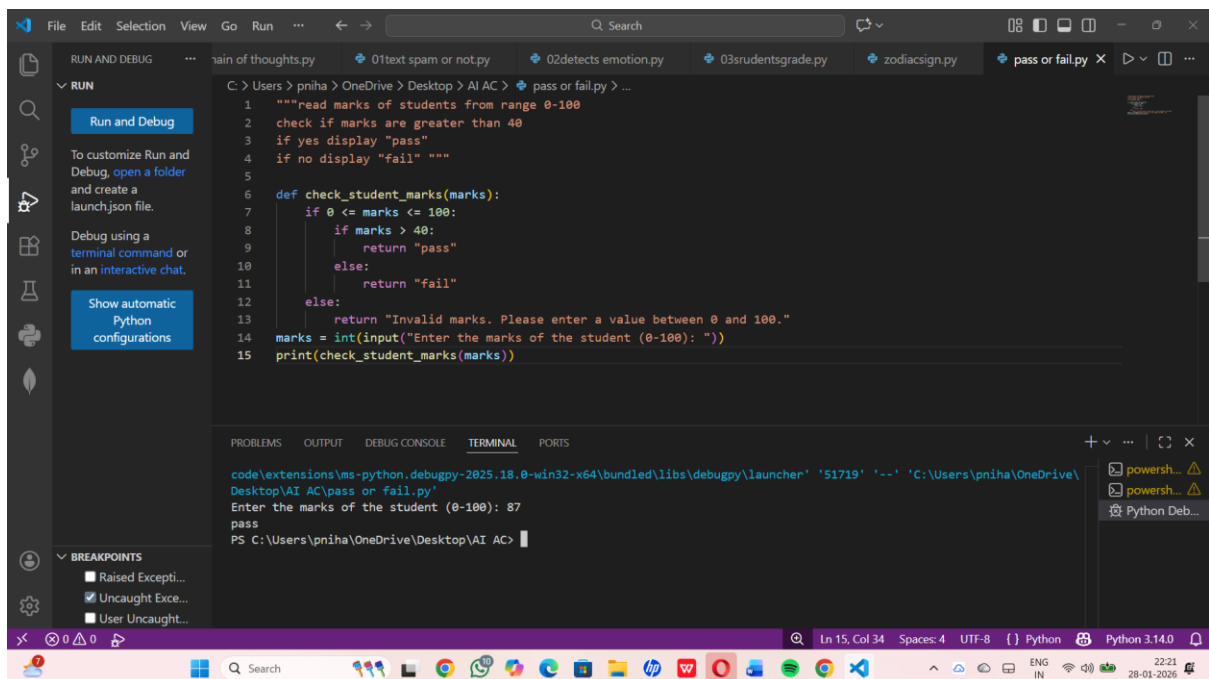


Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student Passes or Fails based on marks using Chain-of-Thought (CoT) prompting.

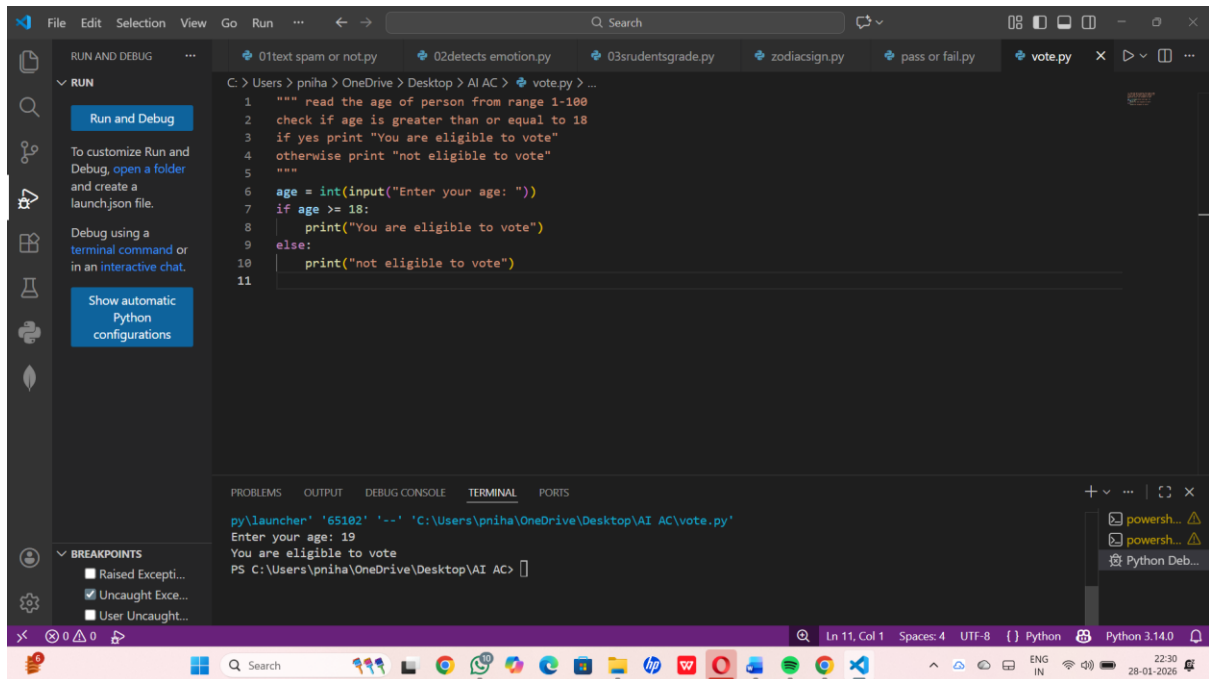
Result Categories:

['Pass', 'Fail']



Q6 Voting Eligibility Check (Chain-of-Thought Prompting) Task:

Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.



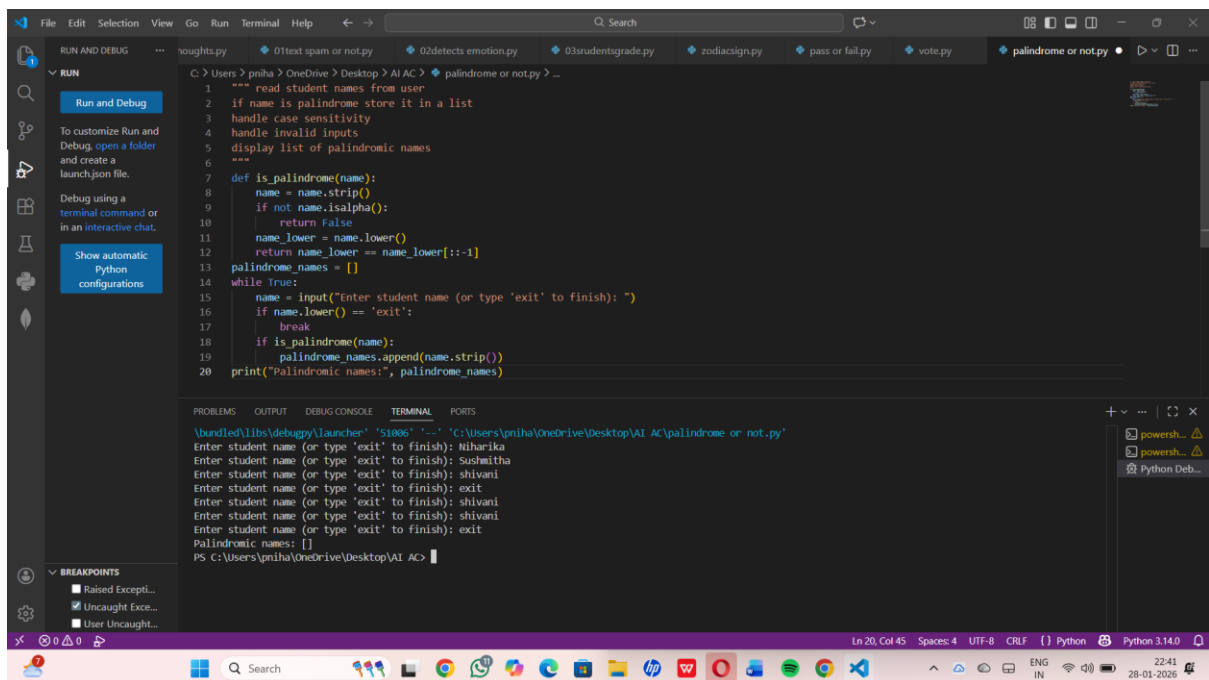
The screenshot shows a Visual Studio Code editor with a Python file named `vote.py`. The code is as follows:

```
1 """ read the age of person from range 1-100
2 check if age is greater than or equal to 18
3 if yes print "You are eligible to vote"
4 otherwise print "not eligible to vote"
5 """
6 age = int(input("Enter your age: "))
7 if age >= 18:
8     print("You are eligible to vote")
9 else:
10    print("not eligible to vote")
11
```

The terminal output shows the program being executed with the input `19`, resulting in the output `You are eligible to vote`.

Q7 Prompt Chaining (String Processing – Palindrome Names)

Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.



The screenshot shows a Visual Studio Code editor with a Python file named `palindrome or not.py`. The code is as follows:

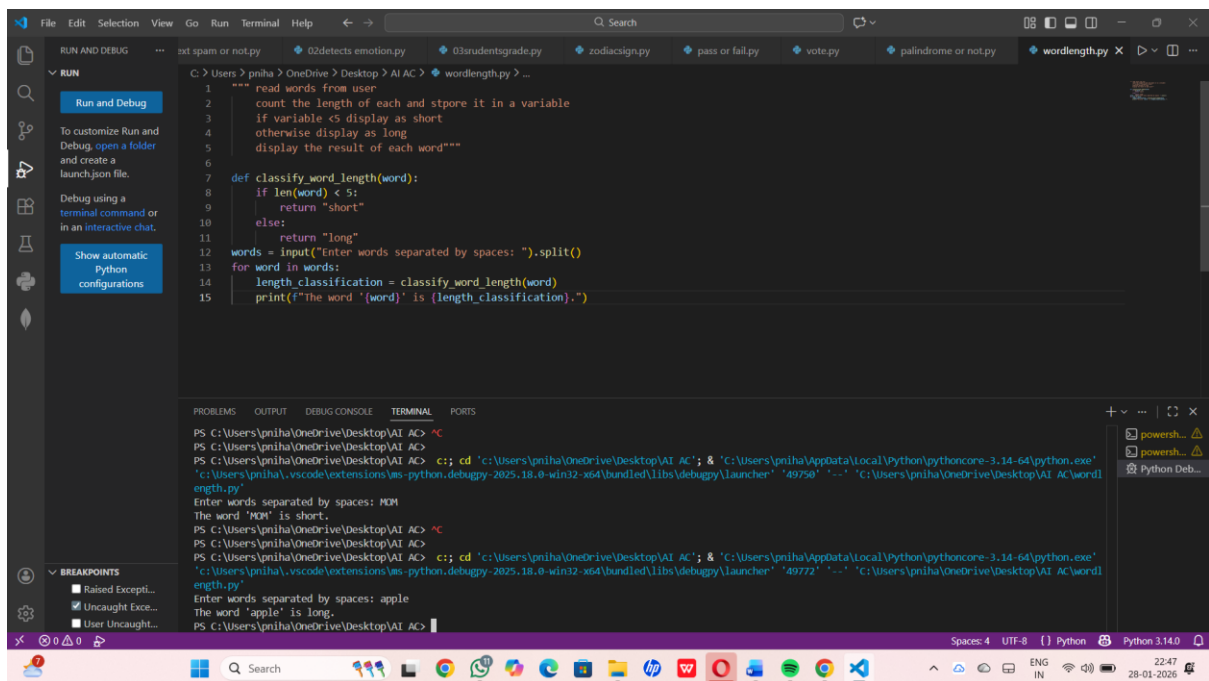
```
1 """ read student names from user
2 if name is palindrome store it in a list
3 handle case sensitivity
4 handle invalid inputs
5 display list of palindromic names
6 """
7 def is_palindrome(name):
8     name = name.strip()
9     if not name.isalpha():
10        return False
11    name_lower = name.lower()
12    return name_lower == name_lower[::-1]
13 palindrome_names = []
14 while True:
15     name = input("Enter student name (or type 'exit' to finish): ")
16     if name.lower() == 'exit':
17         break
18     if is_palindrome(name):
19         palindrome_names.append(name.strip())
20 print("Palindromic names:", palindrome_names)
```

The terminal output shows the program being executed with the input `Niharika`, `Sushmitha`, `shivani`, and `shivani`, resulting in the output `Palindromic names: []`.

Q8 Prompt Chaining (String Processing – Word Length)

Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word



The screenshot shows a Visual Studio Code editor window with a Python file named `wordlength.py` open. The code defines a function `classify_word_length` that takes a word and returns "short" if its length is less than 5, and "long" otherwise. The main program prompts the user to enter words separated by spaces, splits the input into a list, and iterates over each word, printing its classification.

```
1 """ read words from user
2 count the length of each and store it in a variable
3 if variable < 5 display as short
4 otherwise display as long
5 display the result of each word"""
6
7 def classify_word_length(word):
8     if len(word) < 5:
9         return "short"
10    else:
11        return "long"
12 words = input("Enter words separated by spaces: ").split()
13 for word in words:
14     length_classification = classify_word_length(word)
15     print(f"The word '{word}' is {length_classification}.")
```

The terminal output shows the program being executed. The user enters "MM" and "apple". The program outputs "The word 'MM' is short." and "The word 'apple' is long." respectively.

```
PS C:\Users\pnih\OneDrive\Desktop\AI AC> ^C
PS C:\Users\pnih\OneDrive\Desktop\AI AC> ^C
PS C:\Users\pnih\OneDrive\Desktop\AI AC> c:; cd 'C:\Users\pnih\OneDrive\Desktop\AI AC'; & 'C:\Users\pnih\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'C:\Users\pnih\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49750' '-' 'C:\Users\pnih\OneDrive\Desktop\AI AC\wordl
length.py'
Enter words separated by spaces: MM
The word 'MM' is short.
PS C:\Users\pnih\OneDrive\Desktop\AI AC> ^C
PS C:\Users\pnih\OneDrive\Desktop\AI AC> ^C
PS C:\Users\pnih\OneDrive\Desktop\AI AC> c:; cd 'C:\Users\pnih\OneDrive\Desktop\AI AC'; & 'C:\Users\pnih\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'C:\Users\pnih\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49772' '-' 'C:\Users\pnih\OneDrive\Desktop\AI AC\wordl
length.py'
Enter words separated by spaces: apple
The word 'apple' is long.
PS C:\Users\pnih\OneDrive\Desktop\AI AC> |
```